

Guía de Colaboración Técnica

Proyecto: ProtectPyME [cite: 2]

Este manual establece los lineamientos técnicos y de flujo de trabajo para asegurar el desarrollo eficiente del proyecto ProtectPyME [cite: 3]. Es obligatorio para todos los miembros del equipo seguir estas instrucciones para evitar conflictos de código y pérdida de información [cite: 4].

1. Configuración de Cuenta GitHub [cite: 6]

La cuenta de GitHub es nuestra base de operaciones. Debe ser profesional y estar correctamente vinculada [cite: 7].

- **Registro:** Entrar a github.com y registrarse preferentemente con la cuenta de Google [cite: 8].
- **Verificación:** Es indispensable verificar el correo electrónico para poder realizar commits [cite: 9].
- **Nombre de usuario:** Elegir un username profesional [cite: 10].

 **MAL:** juan123xd [cite: 11]

 **BIEN:** juanperez-dev [cite: 14]

2. Instalación de Herramientas [cite: 12]

Indispensable para todos [cite: 13]:

- **Git:** Descargar del sitio oficial e instalar con opciones por defecto [cite: 15].

Para React Native [cite: 16]:

- **Node.js** [cite: 17] y **VSCode** [cite: 18].

Para Unity [cite: 19]:

- **Unity Hub** [cite: 20] y la **versión exacta del proyecto** para evitar errores de compatibilidad [cite: 22].

3. Preparación del Proyecto

Clonación [cite: 24]

```
git clone [URL_DEL_REPO] [cite: 26]
cd ProtectPyME [cite: 27]
```



Configuración Pro: Ramas (Develop)

Una vez clonado, antes de tocar nada, es obligatorio moverse a la rama de desarrollo:

```
git fetch origin
git checkout develop
```

Apertura [cite: 28]

- **Unity:** Abrir la carpeta de Unity desde el Hub [cite: 29].
- **React Native:** Abrir protectpyme-app [cite: 31] y ejecutar `npm install` [cite: 33].

4. Flujo de Trabajo Diario [cite: 34]



REGLA DE ORO [CITE: 36]

SIEMPRE antes de trabajar: `git pull` [cite: 37]. Evita sobrescribir el trabajo de otros [cite: 38].

Pasos Correctos [cite: 39]:

1. **Descargar cambios:** `git pull` [cite: 41].
2. **Hacer cambios:** Scripts, escenas, frontend o backend [cite: 42].

3. **Ver estado:** `git status` [cite: 44].
4. **Guardar:** `git add .` [cite: 48].
5. **Commit:** `git commit -m "Descripción"` [cite: 50].
6. **Subir:** `git push` [cite: 52].

Calidad de Commits [cite: 53, 54]

 **MAL:** "cambios", "update" [cite: 55]

 **BIEN:** "Agrega login", "Corrige bug" [cite: 55]

5. Reglas del Equipo [cite: 58]

COMUNICACIÓN [CITE: 59]

Avisar en WhatsApp antes de archivos críticos: "Voy a modificar login" [cite: 60].

UNITY [CITE: 61]

NO modificar la misma escena al mismo tiempo [cite: 62]. Genera conflictos difíciles [cite: 63].

FRECUENCIA [CITE: 66]

Push pequeños y frecuentes. No esperes semanas [cite: 66].

Roles y Áreas [cite: 65]

Persona	Área
Luz	Backend + Integración [cite: 66]
Compañera 1	Frontend [cite: 66]
Compañera 2	UI Unity [cite: 66]
Compañera 3	Assets [cite: 66]

| 6. Configuración Obligatoria Unity [cite: 73]

En Unity: **Edit** → **Project Settings** → **Editor** [cite: 75].

- **Version Control:** Visible Meta Files [cite: 76].
- **Asset Serialization:** Force Text [cite: 77].

Esto permite que Git lea archivos como texto [cite: 78].